

Техническое задание на разработку игры

Краткое описание

Разработать прототип рогалика (rogue-like) в консоли. Игра полностью текстовая, с возможностью чтения пользовательского ввода. Механика игры базируется на классической текстовой игре Rogue с элементами пошагового боя и элементами Dungeon Crawler.

Характеристики

Название: Game Prototype

Жанр: Rogue-like, Dungeon-crawler

Платформа: консоль Windows

Движок: Visual studio C#

Аналоги: Rogue. Look, your loot (<https://poki.com/en/g/look-your-loot>).

Камень-ножницы-бумага

Цели и задачи

Цель проекта: продемонстрировать принципы ООП на примере простой игры. Должны быть реализованы основные механики, такие как бой и путешествие по подземелью. Допускается пропуск 1-2 механик, допускается наличие ошибок в проекте. Допускается разработка домашних заданий на основе проекта (см. раздел дополнительно).

Игровая механика

Что должно быть реализовано в прототипе:

1. Юниты
2. Предметы
3. Система подземелий
4. Механика боя
5. Механика сбора лута
6. Система пользовательского ввода

Описание каждой механики в соответствующем разделе.

ЮНИТЫ

1. Виды. Игрок, вражеский NPC (гоблин)
2. Характеристики Юнитов. Имя, здоровье (текущее, максимальное) и базовый урон. Значения для здоровья (макс): 30 и 18. Базовый урон: 6 и 2.
3. Слоты юнитов. Оружие и броня. Реализовать в виде словаря, названия слотов

- в виде перечисления (только для игрока).
- 4. Инвентарь. Реализуется в виде списка предметов, с возможностью их сортировки и максимальным объёмом 3 предмета.

Предметы

Должно быть реализовано два вида предметов: экипируемые и экономические. У экипируемых предметов должна быть характеристика прочности.

Экипируемые:

1. Виды. Оружие, броня.
2. Виды оружия. Меч.
3. Характеристики оружия. Название. Урон, прочность (текущая, максимальная). Значения, по-умолчанию, 10 и 15.
4. Виды брони. Просто броня (Armour), дополнительных видов на данном этапе не подразумевается.
5. Характеристики брони. Название, защита. Защита - % от урона, который можно заблокировать. По-умолчанию, равно 5. Максимальное значение - 50.

Экономические:

1. Виды. Используемые, неиспользуемые.
2. Виды неиспользуемого. Золото.
3. Характеристики неиспользуемого: название, количество, можно ли стака́ть.
4. Виды используемого. Зелье здоровья, точильный камень.
5. Характеристики используемого. Название, стоимость, количество, можно ли стака́ть.. Зелье здоровья восстанавливает 7 здоровья, точильный камень - 4 единицы прочности. После использования пропадает из инвентаря!
6. Если предмет стака́ется - увеличивается количество, если нет, он добавляется в отдельный слот. Стака́ется только золото!

Система подземелий

1. Комнаты. В каждой комнате - лут, монстр или ничего. Разветвление - вперёд, направо или влево (0, -1 и 1). У каждой комнаты есть название.
2. Игрок выбирает, куда пойти.
3. Как только игрок заходит в комнату, выводится информация. Если есть лут - собирается лут, если есть враг - начинается бой.
4. Одна из комнат - финальная, как только игрок туда приходит - игра заканчивается.
5. Подземелье реализуется с помощью связного списка. Ссылки на следующие комнаты хранятся внутри Dictionary.
6. *Для демо-версии достаточно собрать подземелье из каждого вида комнат, с любым разветвлением, главное - чтобы игрок всегда смог дойти до финала (сделать структуру более линейной)*

Механика боя

1. Бой начинается автоматически, если игрок входит в комнату, и там вражеский NPC.
2. Бой проходит в пошаговом режиме.
3. Игрок должен ввести: камень, ножницы или бумага (или число от 1 до 3, символизирующее каждую опцию). Вывод в консоли с опциями должен быть после каждого хода.
4. Далее свой выбор делает NPC, и в консоли отображается результат: если игрок побил NPC, NPC получает, если игрока побил NPC - игрок получает урон. Если ничья - ничего не происходит. *Текст может быть любым, главное - чтобы игрок понимал, что произошло.*
5. Урон рассчитывается так: базовый урон + урон от оружия - %от урона, поглощаемый бронёй.
6. В случае смерти игрока, игра заканчивается.
7. В случае смерти NPC, выводится результат боя. Лут собирается автоматически с поверженного врага. Затем предлагается ввести следующую точку маршрута.

Механика сбора лута

1. В случае, если игрок вошёл в комнату, и там есть лут, он собирается автоматически.
2. Если игрок убивает монстра, лут собирается автоматически.
3. Если инвентарь заполнен, выводится сообщение "Инвентарь заполнен" и лут не добавляется в инвентарь игрока.
4. Используемые предметы используются после боя автоматически. После использования - пропадают из инвентаря.

Система пользовательского ввода

1. Весь ввод контекстный.
2. Контекстный ввод. Происходит во время боя, либо когда игрок выбирает маршрут. Игрок вводит один из предложенных вариантов. В бою - камень, ножницы, бумага. В случае маршрута - направление.
3. Обязательна проверка ввода на корректность, в случае неправильного ввода - сообщение о неправильном вводе и повтор опций. *Содержание сообщения должно отражать суть ошибки, текст - на усмотрение разработчика.*

Игровой флоу

Для демо-версии должен быть реализован следующий флоу:

1. Вывод "Welcome, player!"
2. В самой игре происходит создание подземелий с помощью фабрики.
3. Вывод "Enter your name"
4. Игрок вводит имя.

5. Выводится "Hello <имя игрока>"
6. Выводится "Entering the Dungeon"
7. Запускается Game Loop. Должен быть реализован отдельной сущностью, инкапсулирующей игровую механику, принимающей ввод и т.д.
8. Выводится "Entered room <название комнаты>".
9. Первая комната пустая!
10. Выводится выбор доступного маршрута, далее ожидается пользовательский ввод.
11. Последующий флоу - в соответствии с описанной механикой.
12. Когда игрок доходит до финала (заходит в последнюю комнату), выводится "Congratulations <имя игрока>" и результат игрока (выводится информация об игроке)

Дополнительно

Часть указанных ниже механик может быть реализована студентами в качестве самостоятельной работы, либо домашних заданий (в случае, если существующих заданий окажется недостаточно):

1. Сделать так, чтобы оружие после каждого удара теряло один пункт прочности. Реализовать использование точильного камня.
2. Сделать так, чтобы броня после каждого получения урона теряла один пункт прочности. Реализовать использование точильного камня.
3. Реализовать события в случае смерти игрока (11 урок).
4. Реализовать минимальное/максимальное значение урона для оружия. Итоговый урон, который может нанести персонаж: базовый урон + $\text{Random}(\text{min}, \text{max})$.
5. Реализовать класс для парсинга пользовательского ввода. Сделать возможность вызова команд в любой момент: -команда <флаг>. Команды: -info (вывод информации об игроке), -inventory <индекс слота>, -go <маршрут>, -quit (выход из игры).
6. Сделать механику выхода из боя. В этом случае, выбрасывается число от 0 до 20. В случае, если выпало число 0 или число, кратное 3, игрок избегает боя и может выбрать следующую точку маршрута. В случае неудачи, игрок получает 2 единицы урона.
7. Реализовать новые виды брони / оружия. Сделать возможность замены существующей экипировки.
8. В игре реализованы фабрика и строитель в виде статических классов. Реализовать абстракции, сделать по две реализации для каждого класса, позволяющие создавать разных юнитов и разные подземелья. Реализация зависит от уровня сложности (easy, medium, hard). Уровень сложности задаётся в начале (через enum) игроком. Реализация юнитов и подземелий на усмотрение студентов.
9. Сделать лут в подземелье в виде Hashset, реализовать несколько предметов. Переопределить GetHashCode() и Equals() для предметов.
10. Исправить найденные ошибки, провести "чистку" кода в соответствии с нотацией (12 урок).
11. Выделить часть механик в интерфейсы (12 урок).
12. ДОПОЛНИТЕЛЬНОЕ АРХИСЛОЖНОЕ ЗАДАНИЕ. С помощью алгоритмов из теории графов сделать проверку подземелий на корректность - наличие маршрута от начальной до конечной точки. Проверка подземелий на наличие

ЦИКЛОВ.